E-commerce System Documentation

Central reference for the Ecommerce website project, covering architecture, stack, authentication, and API design placeholders for the team to extend.

System Blueprint

Project Overview

The Ecommerce website project is intended to provide a complete online shopping experience for customers and a manageable operational platform for internal teams.

Primary user roles

- **Customer** — browses products, manages a cart, authenticates, and places orders.
- **Admin** — manages product data, reviews orders, and oversees operational workflows.

Primary business goal

- Sell products online through a reliable web experience.
- Manage customer accounts and order activity.
- Support secure authentication, maintainable APIs, and deployable infrastructure.

System Architecture

The system is organized into three core delivery areas that map directly to the project lists.

Frontend

- Customer-facing web application built with React or Next.js.
- Handles authentication screens, product browsing, cart management, and checkout experience.
- Integrates with backend APIs for live data and protected actions.

Backend

- FastAPI-based service exposing authentication, product, and order APIs.
- Applies business rules, request validation, and error handling.
- Connects to PostgreSQL for persistent data storage.

Database

- PostgreSQL stores users, products, orders, and other operational data.
- Serves as the source of truth for business entities and transactional records.

DevOps / Platform

- Docker and docker-compose support local builds and integration workflows.

- CI/CD automates build and validation steps.
- Kubernetes and monitoring support deployment and operational readiness.

External integrations

- JWT-based authentication flow for protected API access.
- Future payment, notification, or third-party commerce services can be added as separate integrations.

Tech Stack

- **Node**— backend framework.
- **PostgreSQL** — primary database.
- **React / Next.js** — frontend framework.
- **Docker** — containerization for backend and frontend services.
- **docker-compose** — local multi-service orchestration.
- **CI/CD pipeline** — automated build, test, and delivery workflow.
- **Kubernetes** — deployment and orchestration target.
- **Prometheus** — metrics collection.
- **Grafana** — monitoring dashboards and visualization.

Authentication Flow (JWT)

1. A user submits login credentials from the frontend.
2. The backend validates the credentials against the user record.
3. On success, the backend issues a JWT access token.
4. The frontend stores and sends the token with protected requests.
5. The backend validates the token on each protected API call.
6. If the token is invalid or expired, the request is rejected and the frontend prompts the user to authenticate again.
7. Refresh or token renewal behavior should be defined consistently once the auth design is finalized.

Expected user actions supported by this flow

- Signing in to the application.
- Viewing protected account or order information.
- Performing authenticated checkout and order-related actions.

API Documentation

Auth APIs

Endpoint name:

Path:

Method:

Purpose:

Key request fields:

Key response fields:

Product APIs

Endpoint name:

Path:

Method:

Purpose:

Key request fields:

Key response fields:

Order APIs

Endpoint name:

Path:

Method:

Purpose:

Key request fields:

Key response fields:

Backend flow

E-commerce Platform (Node.js)

Backend Development Documentation - Day 1

Project Setup

Step 1: Initialize Backend

- Created backend folder
- Initialized Node project:
- npm init -y

Installed Packages

Core Dependencies:

- express → Web framework
- mongoose → MongoDB ORM
- dotenv → Environment variables
- cors → Cross-origin requests
- jsonwebtoken → JWT authentication
- bcryptjs → Password hashing

Dev Dependency:

- nodemon → Auto-restart server

Command used:
npm install express mongoose dotenv cors jsonwebtoken bcryptjs
npm install --save-dev nodemon

Folder Structure

```
backend/
├── src/
│   ├── config/       → Database connection
│   ├── controllers/  → Business logic
│   ├── routes/       → API routes
│   ├── models/       → Database schemas
```

```
│   ├── middlewares/   → Authentication logic
│   └── app.js         → Express app setup
│
├── server.js          → Entry point
├── .env               → Environment variables
├── .env.example       → Sample env file
└── package.json
```

Environment Configuration

.env file:

```
PORT=5000
JWT_SECRET=supersecret
DB_URL=mongodb+srv://semmozhi12122005_db_user:Vendora@vendora.ab3r9fe.mongodb.net/?appName=vendora
```

Database Setup

- Used MongoDB Atlas (cloud database)
- Configured connection using mongoose
- Created database connection file in:
- src/config/db.js

Authentication Module

User Model

- name (String)
- email (String, unique)
- password (hashed)

Register API

Endpoint:

POST /api/v1/auth/register

Functionality:

- Accepts name, email, password
- Checks if user exists
- Hashes password using bcrypt
- Saves user to database

- Returns success response (without password)

Login API

Endpoint:
POST /api/v1/auth/login
Functionality:

- Validates email and password
- Compares hashed password
- Generates JWT token
- Returns token for authentication

JWT Authentication

Middleware:

- Extracts token from Authorization header
- Verifies token using JWT_SECRET
- Attaches user info to request

Header format:
Authorization: Bearer

Product Module (CRUD)

Product Model

- name
- description
- price
- stock
- createdBy (User reference)

Product APIs

1. Create Product
2. POST /api/v1/products

- Requires authentication (JWT)

3. Get All Products

4. GET /api/v1/products
5. Get Single Product
6. GET /api/v1/products/
7. Update Product
8. PUT /api/v1/products/
9. Delete Product
10. DELETE /api/v1/products/

API Testing (Postman)

Flow:

1. Register user
2. Login user → get JWT token
3. Use token in Authorization header
4. Test protected routes (product APIs)

Git Workflow Used

- feature/auth-system → develop → main
- Pull request required before merging
- Branch protection rules applied

Issues Faced & Fixes

1. MongoDB Connection Error
- Fixed by using MongoDB Atlas and correct DB_URL
2. Authentication Failed
- Fixed by correcting username/password and encoding special characters
3. Route Not Found
- Fixed by using correct route prefix (/api/v1)
4. req.body undefined
- Fixed by adding express.json() middleware
5. Unauthorized (401)
- Fixed by adding "Bearer" before token
6. GitHub Push Blocked

- Fixed using Pull Request workflow

7. Large PR Issue

- Fixed by adding .gitignore and removing node_modules

Current Status

Backend setup complete
Authentication system working
JWT security implemented
Product CRUD APIs created
Database connected (MongoDB Atlas)
Git workflow implemented

Backend Development Documentation - Day 2

Client-Side Flow (Cart + Order + Automation Testing)

Objective

To extend the backend system by implementing the complete client-side eCommerce flow including Cart Management, Order Processing, and Automated API Testing using Postman.

Features Implemented

1. Cart System

Description:
Implemented a user-specific cart system where authenticated users can add, update, and manage products before placing an order.

Components:

- **Cart Model**
  - Fields:
    - user (ObjectId, required)
    - items (array of products with quantity)
- **Cart Controller**
  - Add to cart
  - Get cart
  - Update item quantity
  - Remove item
  - Clear cart

- **Cart Routes**
  - POST `/api/v1/cart`
  - GET `/api/v1/cart`
  - PUT `/api/v1/cart/:productId`
  - DELETE `/api/v1/cart/:productId`
  - DELETE `/api/v1/cart`

---

2. Order System

Description:
Developed order creation functionality by converting cart items into a finalized order.

Components:

- **Order Model**
  - user (ObjectId)
  - items (product + quantity)
  - totalAmount (calculated)
  - status (pending/completed)
- **Order Controller**
  - Create order from cart
  - Calculate total price
  - Clear cart after order creation
- **Order Routes**
  - POST `/api/v1/orders`

---

3. Authentication Integration

- All cart and order routes are protected using JWT middleware
- User identity is extracted from token (`req.user.id`)
- Ensures only logged-in users can access cart and orders

API Testing (Postman Automation)

Objective:

Automate full backend testing flow using Postman scripts and environment variables.

Automated Flow:

```
Register → Login → Create Product → Add to Cart → Get Cart → Create Order
```

Environment Variables Used:

- baseUrl
- email (dynamic)
- token (JWT)
- productId

Key Automation Features:

1. Dynamic Email Generation
   - Avoids duplicate user errors (409 conflict)
2. Token Chaining
   - JWT token stored and reused automatically
3. Data Chaining
   - Product ID captured and used in cart
4. Automated Assertions
   - Status code validation
   - Response structure validation

Test Results:

- All API requests executed successfully
- All assertions passed
- Full system workflow validated

Issues Faced & Fixes

1. 401 Unauthorized

- Cause: Missing "Bearer" in header
- Fix: Correct Authorization format

---

2. MongoDB Validation Error

- Cause: Missing user reference in cart
- Fix: Correct JWT payload and middleware

---

3. Duplicate User (409 Conflict)

- Cause: Static email in testing
- Fix: Dynamic email generation using timestamp

---

4. Git Issues

- node_modules pushed accidentally
- Fixed using `.gitignore` and removing from tracking

---

5. Merge Conflicts

- Occurred during branch integration
- Resolved manually using correct logic merging

Git Workflow Followed

- feature/client-app → develop → main
- Pull Request-based merging
- Conflict resolution using CLI
- Branch protection rules enforced

Current System Status

Authentication system complete
Product module working
Cart system fully functional
Order system implemented
API automation using Postman
Clean Git workflow maintained

Architechture overview

The backend follows a modular layered architecture:

Client (Frontend)
↓
Routes Layer (Express Routes)
↓
Controllers Layer (Request Handling)
↓
Service Layer (Business Logic)
↓
Database Layer (ORM/Queries)

Components:

- Routes: Define API endpoints
- Controllers: Handle request/response
- Services: Contain core business logic
- Models: Define database schema
- Middleware: Authentication & validation

Tech Stack

You don't have access to this Doc

- Runtime: Node.js
- Framework: Express.js
- Database: PostgreSQL (or MongoDB)
- ORM: Sequelize / Prisma / Mongoose
- Authentication: JWT (JSON Web Tokens)
- API Testing: Postman
- Containerization: Docker

Project Folder Structure:

```
backend/
 │
 ├──── src/
 │   ├──── config/
 │   ├──── controllers/
 │   ├──── routes/
 │   ├──── services/
 │   ├──── models/
 │   ├──── middlewares/
 │   ├──── utils/
 │   └──── app.js
 │
 ├──── server.js
 ├──── package.json
 ├──── Dockerfile
 └──── .env
```

Authentication Flow

JWT is used for secure authentication.

Flow:

1. User registers or logs in
2. Server validates credentials
3. Generates JWT token
4. Token sent to client
5. Client includes token in Authorization header
6. Middleware verifies token for protected routes

Security:

- Passwords hashed using bcrypt
- Tokens have expiration time

Api end points

Overview

This backend provides a complete eCommerce API system including authentication, product management, cart handling, and order processing.
Base URL:

```
http://localhost:5000/api/v1
```

---

1. Authentication APIs

Register User

Endpoint:

```
POST /auth/register
```

Description:
Registers a new user in the system.
Request Body:

```
{
  "name": "Sem",
  "email": "sem@gmail.com",
  "password": "123456"
}
```

Response:

```
{
  "success": true,
  "message": "User registered successfully",
  "user": {
    "id": "user_id",
    "email": "sem@gmail.com"
  }
}
```

Login User

Endpoint:

```
POST /auth/login
```

Description:
Authenticates user and returns JWT token.
Request Body:

```
{
  "email": "sem@gmail.com",
```

```
  "password": "123456"
}
```

Response:

```
{
  "success": true,
  "token": "JWT_TOKEN"
}
```

---

2. Product APIs

Create Product

Endpoint:

```
POST /products
```

Description:
Creates a new product (requires authentication).

Headers:

```
Authorization: Bearer <token>
```

Request Body:

```
{
  "name": "Laptop",
  "description": "Gaming laptop",
  "price": 1200,
  "stock": 5
}
```

Response:

```
{
  "success": true,
  "product": {
    "_id": "product_id",
    "name": "Laptop"
  }
}
```

Get All Products

Endpoint:

```
GET /products
```

Description:
Fetch all available products.

Get Single Product

Endpoint:

```
GET /products/:id
```

Description:

Fetch product details by ID.

Update Product

Endpoint:

```
PUT /products/:id
```

Description:
Update product details.

Delete Product

Endpoint:

```
DELETE /products/:id
```

Description:
Delete a product.

---

3. Cart APIs

Add to Cart

Endpoint:

```
POST /cart
```

Description:
Adds a product to the user’s cart.

Headers:

```
Authorization: Bearer <token>
```

Request Body:

```
{
  ”productId”: ”product_id”,
  ”quantity”: 2
}
```

Get Cart

Endpoint:

```
GET /cart
```

Description:
Fetch current user cart.

Update Cart Item

Endpoint:

```
PUT /cart/:productId
```

Remove Item

Endpoint:

```
DELETE /cart/:productId
```

Clear Cart

Endpoint:

```
DELETE /cart
```

---

4. Order APIs

Create Order

Endpoint:

```
POST /orders
```

Description:
Creates an order from cart items.

Headers:

```
Authorization: Bearer <token>
```

Response:

```
{
  "success": true,
  "order": {
    "totalAmount": 1000
  }
}
```

---

5. Health API

System Health Check

Endpoint:

```
GET /health
```

Description:
Returns server status and uptime.

Response:

```
{
  "status": "OK",
  "uptime": 120,
  "timestamp": "2026-05-01T..."
}
```

---

6. Rate Limiting

Applied to all `/api` routes

- Prevents abuse
- Returns:

```
{
  "success": false,
  "message": "Too many requests, try again later"
}
```

---

7. Request Tracking

Every request includes:

```
X-Request-ID: REQ-xxxx
```

Used for:

- Debugging
- Log tracing
- Monitoring

---

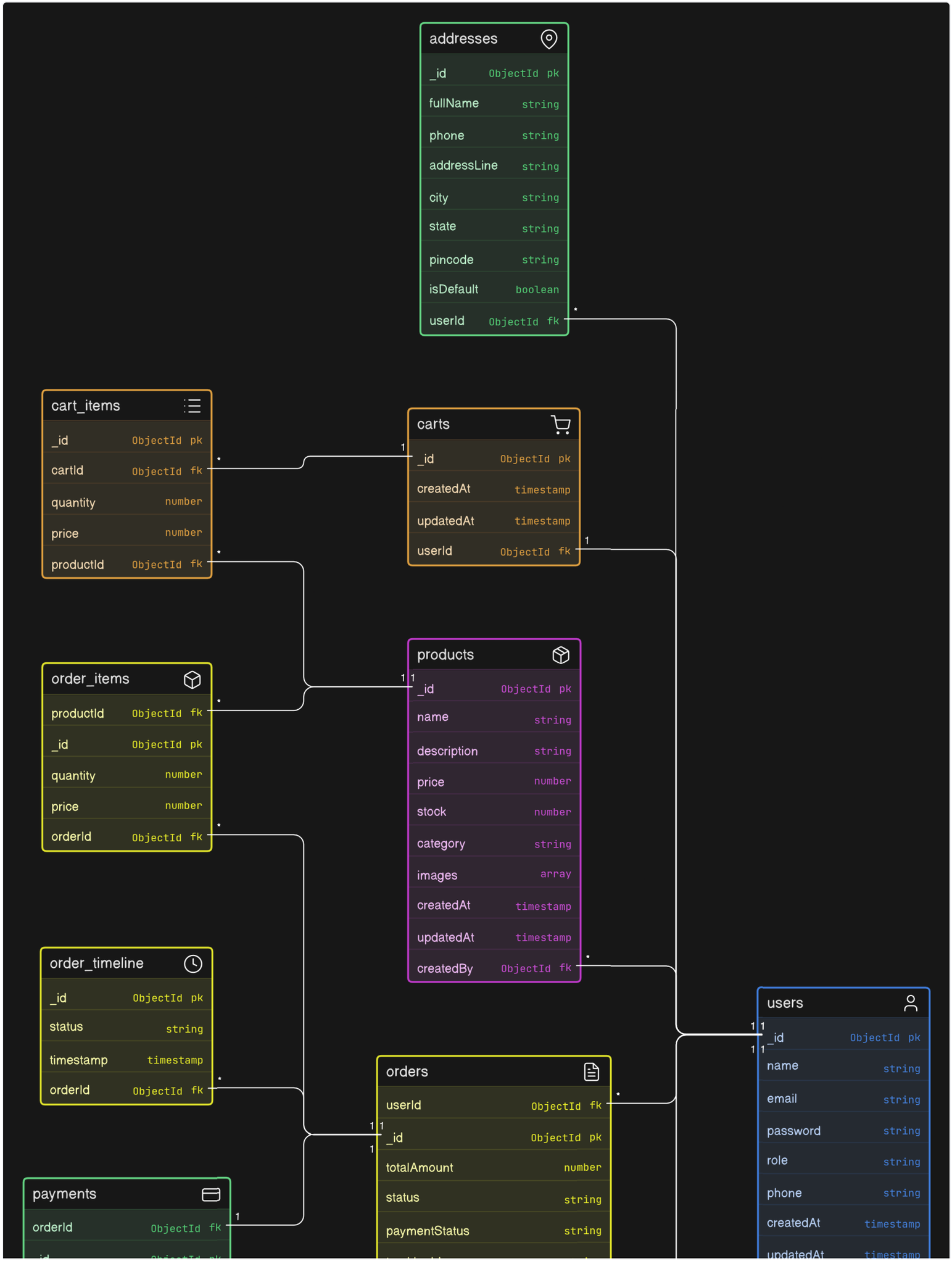
8. Logging System

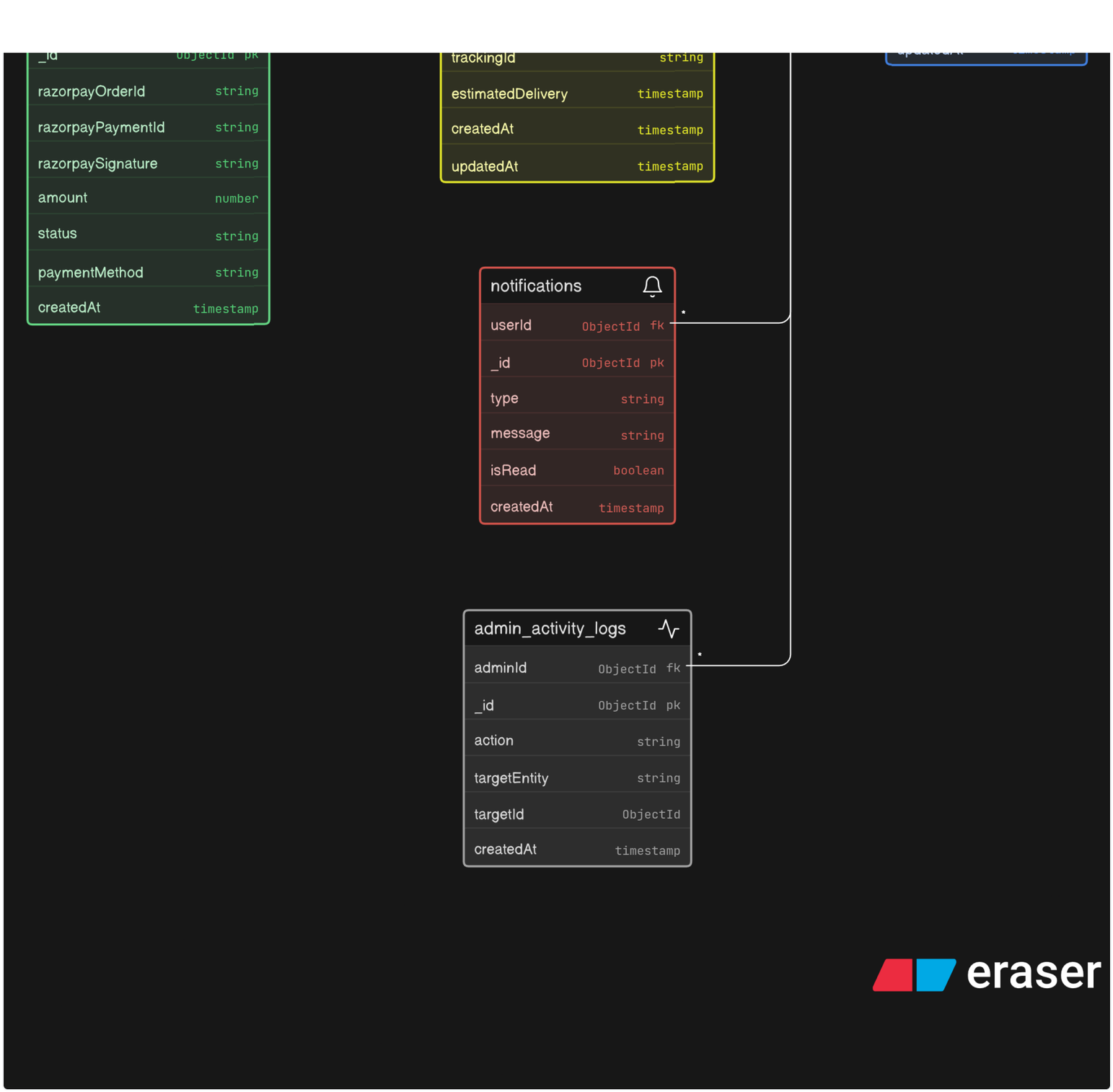
- Uses structured logging
- Logs stored in:
  - logs/app.log
  - logs/error.log

Error Format

```
{
  "success": false,
  "message": "Error message"
}
```

Database design


addresses
_id ObjectId pk
fullName string
phone string
addressLine string
city string
state string
pincode string
isDefault boolean
userId ObjectId fk
cart_items
_id ObjectId pk
cartId ObjectId fk
quantity number
price number
productId ObjectId fk
carts
_id ObjectId pk
createdAt timestamp
updatedAt timestamp
userId ObjectId fk
order_items
productId ObjectId fk
_id ObjectId pk
quantity number
price number
orderId ObjectId fk
products
_id ObjectId pk
name string
description string
price number
stock number
category string
images array
createdAt timestamp
updatedAt timestamp
createdBy ObjectId fk
order_timeline
_id ObjectId pk
status string
timestamp timestamp
orderId ObjectId fk
users
_id ObjectId pk
name string
email string
password string
role string
phone string
createdAt timestamp
updatedAt timestamp
orders
userId ObjectId fk
_id ObjectId pk
totalAmount number
status string
paymentStatus string
payments
orderId ObjectId fk


_id ObjectId pk
razorpayOrderId string
razorpayPaymentId string
razorpaySignature string
amount number
status string
paymentMethod string
createdAt timestamp
trackingId string
estimatedDelivery timestamp
createdAt timestamp
updatedAt timestamp
notifications
userId ObjectId fk
_id ObjectId pk
type string
message string
isRead boolean
createdAt timestamp
admin_activity_logs
adminId ObjectId fk
_id ObjectId pk
action string
targetEntity string
targetId ObjectId
createdAt timestamp
eraser

Error Handling

Standard HTTP status codes are used:

- 200: Success
- 201: Created
- 400: Bad Request
- 401: Unauthorized
- 404: Not Found
- 500: Internal Server Error

Errors are returned in JSON format.

Deployment & Production

The backend is containerized using Docker.

Steps:

1. Create Dockerfile
2. Build Docker image
3. Run container
4. Connect to database
5. Deploy using Kubernetes (future)

CI/CD pipelines automate build and deployment.

docs/
│
├── backend.md
├── docker.md
├── kubernetes.md
├── jenkins.md
├── monitoring.md (Grafana + Prometheus)
└── ansible.md

Docker file

Docker Configuration

Docker is used to containerize the backend service to ensure consistency across environments.

Objectives:

- Package application with dependencies
- Enable portability
- Simplify deployment

Components:

- Dockerfile
- docker-compose.yml

Workflow:

1. Build Docker image
2. Run container
3. Expose application port
4. Connect to database service

Benefits:

- Environment consistency
- Easy scaling
- Faster deployments

Jenkins

CI/CD Pipeline (Jenkins)

Jenkins automates the build and deployment process.

Pipeline Stages:

1. Code Checkout (GitHub)
2. Install Dependencies
3. Run Tests
4. Build Docker Image
5. Push to Docker Registry
6. Deploy to Kubernetes

Benefits:

- Automated workflows
- Continuous integration
- Faster releases

Kubernetes

Kubernetes Deployment

Kubernetes is used to orchestrate containerized applications.

Components:

- Pods: Run containers
- Deployments: Manage replicas
- Services: Expose application
- Ingress: Manage external access

Workflow:

1. Deploy Docker container as Pod
2. Create Deployment for scaling
3. Expose using Service
4. Configure Ingress for routing

Benefits:

- Auto-scaling
- High availability
- Load balancing

Monitoring

Monitoring & Observability

Monitoring is implemented using Prometheus and Grafana.

Prometheus:

- Collects metrics from services
- Stores time-series data

Grafana:

- Visualizes metrics
- Dashboards for system monitoring

Metrics Monitored:

- CPU usage
- Memory usage
- Request latency
- Error rates

Benefits:

- Real-time insights
- Performance tracking
- Alerting capabilities

Ansible

Configuration Management (Ansible)

Ansible is used to automate infrastructure setup and configuration.

Use Cases:

- Install dependencies
- Configure servers
- Setup monitoring tools
- Deploy services

Workflow:

1. Define playbooks
2. Configure inventory
3. Execute automation tasks

Benefits:

- Agentless automation
- Repeatable setup
- Reduced manual errors

Git Workflow & Branch Management

Git Workflow & Branch Strategy

This project follows a structured Git workflow to ensure clean code management, safe collaboration, and production stability.

Branch Structure

The repository uses the following branches:

- main → Production-ready code
- develop → Integration and testing branch
- feature/* → Used for developing new features
- bugfix/* → Used for fixing bugs in development
- hotfix/* → Used for urgent fixes in production
- devops → Infrastructure and deployment configurations

Branch Purpose

main:
Contains stable and production-ready code. No direct commits are allowed.
develop:
Acts as the main development branch where all features are merged and tested before release.
feature/*:
Each new feature is developed in a separate branch created from develop.
bugfix/*:
Used to fix bugs identified during development.
hotfix/*:
Used for critical fixes directly in production (main branch).
devops:
Contains configurations related to Docker, Kubernetes, CI/CD, and infrastructure.

git comment offen use:
git add .
git commit -m "feat: initial backend setup"
git push origin develop

Development Workflow

Development Workflow

The project follows a pull request-based workflow:

1. Create a feature branch from develop
2. Implement the feature
3. Commit and push changes
4. Create a pull request (PR) to develop
5. Review and merge into develop
6. After testing, merge develop into main for production

Example Flow:

feature/auth-system → develop → main

Pull Request (PR) Process

Pull Request Process

A Pull Request (PR) is used to propose changes from one branch to another.

Steps:

1. Push feature branch to GitHub
2. Create PR targeting develop
3. Add description and details
4. Review code changes
5. Merge after approval

PR Title Format:

[type]: short description
Examples:

- feat: add authentication system
- fix: resolve login issue
- docs: update documentation

Branch Protection Rules

Protected Branches

Main Branch

- Production-ready code
- No direct commits allowed
- Only updated via Pull Requests

Develop Branch

- Integration branch for features
- All features merged here before production
- Requires PR approval

Production Branch

- Used for deployment
- Strictly protected
- No direct changes allowed

Staging Branch

- Pre-production testing environment
- Used for validating features before production

Rules Configured

1. Restrict Direct Push

- Developers cannot push directly to protected branches
- All changes must go through Pull Requests

2. Require Pull Request Before Merge

- Every change must be reviewed via PR
- Prevents unverified code from entering main branches

3. Required Approvals

- Minimum 1 approval required before merge
- Ensures peer review

---

4. Require Branch to Be Up-to-Date

- Feature branch must sync with target branch before merging
- Prevents merge conflicts in main branches

---

5. Require Conversation Resolution

- All PR comments must be resolved before merge
- Ensures code issues are addressed

---

6. Block Force Push

- Prevents rewriting history of protected branches
- Ensures commit history integrity

---

7. Require Linear History

- Disables merge commits
- Encourages clean commit history (rebase/squash)

Workflow Followed

```
Feature Branch → Develop → Main → Production
```

Step-by-Step Flow:

1. Create feature branch
2. → `feature/client-app`
3. Develop and commit changes
4. Push feature branch
5. Create Pull Request → `develop`
6. Review and approve
7. Merge into develop
8. Later merge develop → main

Issues Faced

Push Rejected (GH013)

- Cause: Direct push to protected branch
- Solution: Use Pull Request workflow

Merge Blocked

- Cause: Approval required
- Solution: Approve PR before merging

Cannot Push After Conflict Fix

- Cause: Protected branch rules
- Solution: Create temporary branch and PR

Best Practices

- Always work on feature branches
- Never commit directly to main
- Keep commits small and meaningful
- Use descriptive commit messages
- Resolve conflicts locally before PR

Benefits

- Improved code quality
- Reduced production bugs
- Structured collaboration
- Clean Git history
- Industry-standard workflow

Release Strategy

Release Strategy

Releases are managed through the following flow:

1. All features are merged into develop
2. Testing is performed on develop
3. develop is merged into main
4. Production deployment is triggered

This ensures only stable and tested code reaches production.

Project Initialization

Create Root Project

```
mkdir opscart
cd opscart
git init
```

Create Professional Folder Structure

```
mkdir backend frontend docs infra .github
mkdir infra/docker infra/kubernetes infra/ansible infra/ci-cd
mkdir .github/workflows
```

Final Structure

```
opscart/
 │
 ├──── backend/
 ├──── frontend/
 ├──── docs/
 ├──── infra/
 │     ├──── docker/
 │     ├──── kubernetes/
 │     ├──── ansible/
 │     └──── ci-cd/
 │
 ├──── .github/
 │     └──── workflows/
 │
 ├──── README.md
 ├──── .env.example
 ├──── .gitignore
 └──── docker-compose.yml
```

Initialize Node Backend

```
cd backend
npm init -y
npm install express dotenv cors mongoose jsonwebtoken bcryptjs
npm install --save-dev nodemon
```

Backend Structure

```
backend/
 │
 ├──── src/
 │     ├──── config/
 │     ├──── controllers/
 │     ├──── routes/
 │     ├──── services/
 │     ├──── models/
 │     ├──── middlewares/
 │     ├──── utils/
 │     └──── app.js
 │
 ├──── server.js
 ├──── package.json
 └──── .env
```

Payment Integration(Razorpay)

This project integrates a payment gateway using Razorpay (Test Mode) to simulate real-world transaction processing in an eCommerce system.
The goal is to implement a secure and production-like payment workflow without using real money.

Objective

- Simulate real-time payment flow
- Integrate secure payment verification
- Connect payment system with order management
- Demonstrate production-level backend architecture

Key Concept

Razorpay is used only for payment processing.
All business logic such as order creation, validation, and status updates are handled by the backend.

Payment Flow Architecture

```
User → Checkout → Backend → Razorpay → Payment → Verification → Order Update
```

System Workflow

Create Order (Backend)

- Endpoint:

```
/api/v1/payment/create-order
```

- Backend sends:
  - amount
  - currency
- Razorpay returns:
  - order_id

Initiate Payment (Frontend)

- Razorpay Checkout UI is opened
- User completes payment (Test Mode)

Payment Response

Razorpay returns:

- payment_id
- order_id
- signature

Verify Payment (Backend)

- Endpoint:

```
/api/v1/payment/verify
```

- Backend validates:
  - order_id
  - payment_id
  - signature

Update Order Status

- If verification successful:

```
order.status = "PAID"
```

- Else:

```
order.status = "FAILED"
```

Security Considerations

Backend Verification Required

- Payment success must NOT be trusted from frontend
- Signature verification is mandatory

Key Management

Key
Usage
RAZORPAY_KEY_ID
Used in frontend
RAZORPAY_KEY_SECRET
Used in backend only

Key	Usage
RAZORPAY_KEY_ID	Used in

	frontend
RAZORPAY_KEY_SECRET	Used in backend only

Security Rules

- Never expose secret key to frontend
- Always verify payment signature
- Validate order amount before processing

Test Mode Configuration

- Razorpay Test Mode is used
- No real transactions occur
- Safe for development and testing

Test Payment Details

- Card Number: 4111 1111 1111 1111
- Expiry: Any future date
- CVV: 123
- OTP: 123456

Backend Structure

```
/payment
  ├──── payment.controller.js
  ├──── payment.routes.js
  ├──── payment.service.js
```

API Endpoints

Create Razorpay Order

```
POST /api/v1/payment/create-order
```

Verify Payment

```
POST /api/v1/payment/verify
```

Integration with Existing System

```
Cart → Order Created (PENDING)
       ↓
Payment Process
```

```
        ↓
Verification
        ↓
Order Updated (PAID)
```

Common Mistakes Avoided

- Trusting frontend payment success
- Exposing secret keys
- Skipping signature verification
- Directly marking orders as paid

ADMIN PANEL DEVELOPMENT

Admin Dashboard

Implemented:

- Sidebar navigation
- Dashboard overview
- Statistics cards
- Recent orders table

Design Style:

- Apple-inspired minimal UI
- Clean spacing
- White/black design system

Admin Products Page

Implemented Planning:

- Product table
- Create product modal
- Edit product flow
- Delete product flow
- CRUD UI structure

Admin Orders Page

Implemented Planning:

- Orders table
- Status dropdown
- Timeline-ready order structure
- Order management flow

Admin Users Page

Implemented Planning:

- User listing

- Role management
- Role update dropdown
- Self-role protection

Admin Route Protection

Implemented:

- AdminRoute component
- Role-based route protection
- Unauthorized redirects

CLIENT FRONTEND DEVELOPMENT

Frontend Architecture

Implemented:

- React structure
- Routing system
- Axios setup
- Auth context
- Protected routes

Home Page Planning

Designed:

- Hero section
- Product listing
- Ecommerce video sections
- Hover image interactions
- Featured products
- CTA sections

Design Direction:

- Premium Apple-style UI
- Minimal luxury aesthetic
- Smooth interactions

Product Details Page Planning

Designed:

- Product gallery
- Image zoom
- Hover effects
- Related products
- Add to cart integration

Cart Page Planning

Designed:

- Quantity updates
- Remove item flow
- Dynamic total pricing
- Checkout CTA

Checkout Page Planning

Designed:

- Address form
- Razorpay integration
- Order summary
- Payment verification
- Success flow

Profile System Planning

Designed:

- Account overview
- My orders
- My details
- Change password
- Address book
- Preferences
- Logout system

AUTHENTICATION SYSTEM

Auth Context

Implemented:

- Login persistence
- User restoration
- LocalStorage integration
- Logout handling

Protected Routes

Implemented:

- ProtectedRoute
- AdminRoute
- Redirect logic

Login System

Implemented:

- Login UI
- Backend integration
- Token handling
- Role-based redirects

Register System

Planned:

- Registration UI
- Validation system
- Auto-login flow
- Backend integration

NOTIFICATION SYSTEM

Email Notification Planning

Planned:

- Order placed email
- Payment success email
- Shipped email
- Delivered email

Tech Stack:

- Nodemailer
- SMTP integration

DEVOPS & PRODUCTION READINESS

Infrastructure Planning

Planned:

- Dockerization
- AWS EC2 deployment
- NGINX reverse proxy
- HTTPS setup
- CI/CD pipeline

Monitoring & Reliability

Implemented/Planned:

- Logging system
- Request IDs
- Health checks
- Graceful shutdown
- Timeout handling
- Rate limiting

Future Enhancements

Planned:

- Redis caching
- Queue systems
- Notification queues
- Analytics dashboard
- CDN optimization
- Kubernetes readiness

BRANCHING STRATEGY

You don't have access to this Doc

Recommended Git Branch Structure

Main Branches

- main
- develop

Feature Branches

- feature/auth-system
- feature/admin-panel
- feature/client-home
- feature/cart-checkout
- feature/order-tracking
- feature/notifications
- feature/profile-system
- feature/devops